

EventForce Management System

Phase 1: Requirement Analysis and Planning

Updated 5 October 2026 | Team SWUID-2026-379743

KSK College of Engineering and Technology | College code: 8210

Team: Srinivasan J, Harish V and Subikshan C

Problem and user story

Event coordinators need one system for client requests, venue availability, vendor assignments and feedback. We designed EventForce to reduce scattered booking information and make cancellation decisions traceable.

Ideation and customer journey

The design persona prioritizes accurate bookings and clear ownership. The journey is client registration, event planning, venue selection, vendor assignment, confirmation, event execution and feedback. Cancellation branches into approval or rejection. This is a design model, not an interview study.

Functional requirements

Maintain Event, Client, Vendor, Venue, Feedback and Event Vendor records; prevent venue/date conflicts; support cancellation approval; send cancellation and reminder notifications; complete past events on a schedule; restrict access by responsibility; report on upcoming events.

Nonfunctional requirements

Preserve related-record integrity, validate required booking information, support bulk Apex operations and provide understandable errors. No production performance benchmark or uptime guarantee has been measured.

Architecture and data model

Lightning app and custom tabs form the interface. Event looks up Client and Venue. Feedback looks up Event and Client. Event Vendor links Event and Vendor through two master-detail relationships. Flows, approvals and Apex provide automation; reports and dashboards provide operational views.

Plan and acceptance criteria

Build the data model first, then booking logic, cancellation, reminders, UI, sample data, security and regression tests. Evidence must distinguish saved configuration from runtime outcomes. Inbox delivery and complete role/sharing coverage remain acceptance checks; selected coordinator actions have been verified.

Design model: Empathy

This design persona represents a coordinator managing several events. It is based on the project problem statement, not on a claimed interview study.

Event coordinator empathy map

THINK & FEEL

Will the venue be free? Have all client changes been recorded?

SEE

Separate client messages, changing dates and vendor commitments.

HEAR

Client requests, vendor updates and approval decisions.

SAY & DO

Confirm details, assign vendors and follow up on pending work.

PAINS

Duplicate entries, missed reminders and conflicting bookings.

GAINS

Linked records, reliable checks and a clear decision history.

The response to these needs is concrete: connected records reduce repeated entry; venue/date validation handles conflicts; scheduled reminders support communication; Approval History preserves cancellation decisions.

Design model: Ideation

We compared possible approaches against coordination, automation and maintenance needs. The comparison is qualitative; no numeric user-study scores are invented.

From problem to selected solution

Manual registers

Low setup effort; repeated entry and limited shared visibility.

Spreadsheets

Useful for lists; related workflows and controlled approvals require extra work.

Custom standalone portal

Flexible interface; adds application and hosting maintenance.

Salesforce solution

Related data, Flow, Apex, permissions and reporting in one platform.

Prioritize first

Booking integrity, relationships and controlled cancellations.

Prioritize next

Reminders, analytics, feedback and maintainable documentation.

Salesforce was selected because the assigned project is intended to demonstrate CRM configuration and development. A client self-service portal remains future work; it is not part of the demonstrated implementation.

Design model: Requirements

This model connects the main business requirements to the Salesforce component responsible for them.

Requirements mapped to components

Data management

Six custom objects and explicit lookup/master-detail relationships.

Booking integrity

EventForceService validates venue/date combinations.

Cancellation control

Approval process sets Canceled or Rejected outcomes.

Communication

Reminder flow and client cancellation notification flow.

Operational oversight

Upcoming Events report and Operations Dashboard.

Access control

Profiles, roles, permission set and sharing configuration.

The mapping supports acceptance testing. For each requirement, the team can identify an object, automation component or access rule and then choose an observable test. Configuration presence and runtime behavior are tracked as separate forms of evidence.

Design model: Roadmap

The roadmap follows the guide while ordering dependent work before its consumers. It is a work sequence, not a claim of measured sprint dates or individual contributions.

Five-phase development roadmap

1. Requirements

Define objects, stakeholders, relationships and acceptance scenarios.

2. Backend

Build fields, formulas, validation, Apex, approvals and flows.

3. User experience

Create tabs, Lightning app, layouts, report and dashboard.

4. Data and security

Import sample data; configure and test controlled access.

5. Documentation

Preserve evidence, source snapshot, report and narration.

Review loop

Resolve discrepancies and rerun affected checks after changes.

The project backlog uses estimated story points only. Actual sprint velocity cannot be calculated from unrecorded dates or unrecorded accepted points. The deliverables are supported by files and screenshots rather than invented planning history.